

高级语言程序设计实验报告

南开大学 计算机大类

黄小璐

2511354

工科试验班模拟 1-3 班

课程号：3137

2026 年 5 月 15 日

目录

高级语言程序设计大作业实验报告	错误！未定义书签。
一. 作业题目	错误！未定义书签。
二. 开发软件	错误！未定义书签。
三. 课题要求	错误！未定义书签。
四. 主要流程	错误！未定义书签。
1. 整体流程	1
2. 类图设计	错误！未定义书签。
(1) 核心类	错误！未定义书签。
(2) 继承关系	错误！未定义书签。
3. 核心算法	错误！未定义书签。
(1) 第一次点击安全布雷算法.....	3
(2) 周围地雷数量计算.....	3
(3) 空白区域展开算法.....	4
五. 单元测试	4
六. 收获与总结	5
七. AI 使用披露.....	6

高级语言程序设计 实验报告

一. 作业题目

扫雷小游戏（包括三种模式+自定义）

二. 开发软件

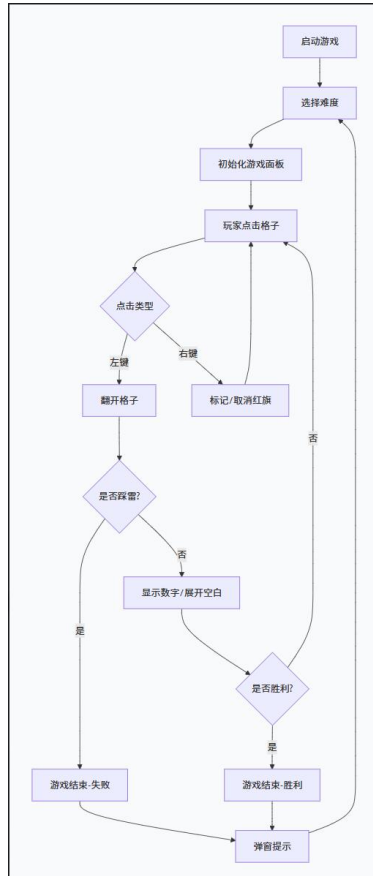
Qt Creator

三. 课题要求

使用 C++ 语言
图形化界面
面向对象设计
单元测试
版本控制
项目讲解视频
AI 工具披露

四. 主要流程

1. 整体流程



2. 类图设计

本项目采用面向对象设计，共包含三个核心类：

MainWindow

m_mineField: MineField
m_mineCountLabel: QLabel
m_difficultyCombo: QComboBox

MainWindow()
~MainWindow() //析构函数
+ setupUI()
+ startGame(rows,cols,mines)
+ onDifficultyChanged()
+ onCustomGame()
+ onGameOver(win)



MineField

m_grid: vector<vector<CellButton>
m_isMine: vector<vector<bool>>
m_adjacentMines: vector<vector<int>>
m_revealed: vector<vector<bool>>
m_flagged: vector<vector<bool>>

MineField()
virtual ~MineField() //虚析构函数
newGame(rows,cols,mines)
placeMines(firstX,firstY)
calculateNumbers()
revealCell(x,y)
checkWin()



CellButton

m_x, m_y: int
m_isMine: bool
m_isRevealed: bool
m_isFlagged: bool
m_adjacentMines: int

CellButton(x,y,parent)
virtual ~CellButton()

```
reset()
mousePressEvent()
signal leftClicked(x,y)
signal rightClicked(x,y)
```

继承关系：

`CellButton` 继承自 `QPushButton`，复用 Qt 按钮组件的点击、样式等功能
`MineField` 继承自 `QWidget`，作为扫雷面板容器
`MainWindow` 继承自 `QMainWindow`，作为主窗口

3. 核心算法

1) 第一次点击安全布雷算法

```
```cpp
void MineField::placeMines(int firstX, int firstY)
{
 while (placed < m_mineCount) {
 int x = rand() % m_rows;
 int y = rand() % m_cols;
 // 避开第一次点击的格子及其周围 8 格
 bool isForbidden = (abs(x-firstX)<=1 && abs(y-firstY)<=1);
 if (!m_isMine[x][y] && !isForbidden) {
 m_isMine[x][y] = true;
 placed++;
 }
 }
}
```
```

设计思路：传统扫雷游戏在开局时就布置所有地雷，可能导致玩家第一步就踩雷。本程序在玩家点击第一个格子后才开始布雷，并且避开该格子及其周围 8 个相邻位置，保证了玩家第一步永远不会踩到地雷，极大提升了游戏体验。

（2）周围地雷数量计算

对于每个非地雷格子，统计其周围 8 个方向（上、下、左、右、四个对角）中的地雷数量：

```
```cpp
for (int di = -1; di <= 1; di++) {
 for (int dj = -1; dj <= 1; dj++) {
 if (di == 0 && dj == 0) continue;
 // 检查边界内是否有地雷
 }
}
```

```
 }
}
...
```

数字 1-4 分别用蓝、绿、红、紫四种颜色显示，增强视觉辨识度。

### （3）空白区域展开算法（队列迭代 vs 递归）

问题：传统递归实现空白区域展开时，若空白区域过大（如高级模式  $16 \times 30$ ），可能导致函数调用栈溢出，程序崩溃。

解决方案：采用 `std::queue` 队列实现迭代式广度优先遍历：

```
```cpp  
std::queue<std::pair<int, int>> toReveal;  
toReveal.push({x, y});  
  
while (!toReveal.empty()) {  
    auto [cx, cy] = toReveal.front();  
    toReveal.pop();  
    // 处理当前格子，将相邻未翻开格子加入队列  
}  
...`
```

优势：

栈空间占用恒定，不会溢出

时间复杂度 $O(n)$ ，与递归相同

更易于理解和调试

五. 单元测试

测试项	输入	预期结果	实际结果
初级模式启动	选择初级	9×9 面板，10 个雷	通过
中级模式启动	选择中级	16×16 面板，40 个雷	通过
高级模式启动	选择高级	16×30 面板，99 个雷	通过
自定义模式	10×10 , 15 雷	面板大小正确，雷数正确	通过
左键点击安全格	点击非雷格	显示数字或空白	通过
左键点击地雷	点击地雷	游戏结束，显示所有雷	通过
右键标记红旗	右键点击未翻格子	显示萝卜，剩余雷数减 1	通过
右键取消红旗	右键点击红旗	取消萝卜，剩余雷数加 1	通过
胜利判定	翻开所有非雷格	弹出胜利对话框	通过
新游戏重置	点击新游戏	面板重置，状态清空	通过
音效开关	点击复选框	胜利/失败音效开关正常	通过
窗口自适应	切换难度	窗口大小自动调整	通过

边界测试

测试场景	输入	预期结果	实际结果
最小自定义	8*8， 1 雷	正常运行	通过
最大自定义	30*40， 1199 雷	正常运行	通过
第一步踩雷保护	点击任意第一格	永远不是雷	通过

六. 收获与总结

1. Qt 信号与槽机制

本项目中，组件间的通信大量使用了 Qt 的信号与槽机制：

```
```cpp
// CellButton 发出信号
emit leftClicked(m_x, m_y);

// MineField 接收信号
connect(btn, &CellButton::leftClicked, this, &MineField::onCellLeftClick);

// MineField 发出信号
emit gameOver(true);

// MainWindow 接收信号
connect(m_mineField, &MineField::gameOver, this, &MainWindow::onGameOver);
```
```

理解：信号与槽实现了组件间的解耦。格子不需要知道是谁在监听它的点击，面板也不需要知道是谁在处理游戏结束事件。这种设计使得各模块可以独立开发、测试和维护，极大地提高了代码的可扩展性。

2. 队列迭代替代递归

在空白区域展开功能中，递归实现简单直观，但在大面积空白区域时容易栈溢出。改用`std::queue`迭代实现后，栈空间占用恒定，程序稳定性显著提升。

3. 资源管理与内存安全

`new`创建的对象需要正确释放。本程序中，切换难度时会先断开信号连接、删除旧格子，再创建新格子，避免了内存泄漏和野指针问题。

```
```cpp
```

```
// 断开信号
disconnect(btn, nullptr, this, nullptr);
// 删除对象
delete btn;
// 设置父对象为 nullptr，避免二次释放
btn->setParent(nullptr);
````
```

七. AI 工具使用披露

本项目的开发过程中使用了以下 AI 工具辅助完成：

| AI 工具名称 | 版本 | 用途 | 使用位置 |
|----------|----|-----------------|------------------|
| DeepSeek | R1 | 信号槽的链接，用于布置雷的位置 | 信号链接 |
| DeepSeek | R1 | 问题诊断 | 编译错误分析、闪退原因排查及修改 |
| DeepSeek | R1 | 设置窗口大小 | 在不同模式切换时调整窗口大小 |
| DeepSeek | R1 | 边界诊断 | 找出边界存在 bug 并修改 |
| DeepSeek | R1 | 代码重构 | 递归→队列迭代改写，避免闪退问题 |
| DeepSeek | R1 | 音效方案调试 | 音效模块实现与问题排查 |

说明：AI 工具主要用于协助代码调试、算法优化建议、错误信息分析。核心逻辑设计、架构规划和功能测试均由本人独立完成。